Vermont Health Platform — Comprehensive Improvement Plan

Version 1.0 | March 2026

> This document covers every dimension of the platform: technical architecture, AI/RAG systems, database, APIs, frontend/UI, security, performance, testing, deployment, domain knowledge, content strategy, mission/values, marketing, business model, and community. Each section includes specific, actionable recommendations. A complete Vercel + Railway deployment HOW-TO guide is included at the end.

TABLE OF CONTENTS

1. AI / RAG / LLM / Chatbot

1.1 Upgrade LLM Model Routing

Current state: The platform is locked to `llama-3.3-70b-versatile` via Groq for all tiers and all query types.

Problem: A simple factual lookup does not need a 70B parameter model. A complex Advisory-tier strategic analysis may benefit from a model more capable than any Groq-hosted open-source model.

Recommendation: Add model-routing logic based on tier and query complexity:

- Free/Subscriber simple Q&A → `llama-3.1-8b-instant` (Groq, very fast, cheap)
- Subscriber/Professional deep analysis → `llama-3.3-70b-versatile` (current, good balance)
- Advisory strategic synthesis → Claude Sonnet 4.6 via Anthropic API (highest quality, justified at custom pricing)

This simultaneously reduces cost for commodity queries and improves quality for premium users.

1.2 Add Hybrid Search (BM25 + Vector)

Current state: The RAG pipeline uses pure vector similarity search against the `rag_documents` pgvector table.

Problem: Pure vector search fails on exact term lookups. Healthcare is acronym-dense — ACO, FHIR, AHEAD, VBP, HCC, SDOH, APM, RHT. A user asking "explain the AHEAD model" may get poor vector retrieval because "AHEAD" as a string is semantically similar to many documents, but only a small number contain the AHEAD All-Payer Claims database specifically.

Recommendation: Implement hybrid retrieval using Reciprocal Rank Fusion (RRF):

1. Run BM25 sparse keyword retrieval on the same documents (add `tsvector` column + GIN index to `rag_documents`)
2. Run vector ANN search (current method)
3. Merge ranked results using RRF before passing to LLM
4. LlamaIndex has a built-in `QueryFusionRetriever` that handles this

This is one of the highest-ROI RAG improvements possible and requires no changes to the LLM or embedding model.

1.3 Add Re-Ranking Step

Current state: Top-N retrieved chunks go directly to the LLM without any re-ranking.

Problem: The top-N by cosine similarity is not necessarily the top-N by relevance to the specific question. Irrelevant chunks consume context window tokens and degrade answer quality.

Recommendation: Add a cross-encoder re-ranker as a post-retrieval step:

- Retrieve top-20 chunks by vector/BM25
- Re-rank with a cross-encoder (Cohere Rerank API, or local FlashRank for zero added cost)
- Pass only top-5 to the LLM

This dramatically improves precision of the final context at very low cost. LlamaIndex supports this via `SentenceTransformerRerank` or `CohereRerank` postprocessors.

1.4 Fix Chunking Strategy

Current state: Documents are truncated at 8,000 characters. PDFs and Sanity content are split by this hard character limit.

Problem: Hard character truncation cuts mid-sentence, breaks logical units, and destroys context. A policy analysis that spans a section boundary is split in a way that makes both halves less useful. This is one of the most common RAG failure modes.

Recommendation: Replace hard truncation with semantic chunking:

- Use `SentenceWindowNodeParser` (LlamaIndex) — splits on sentences, keeps a window of surrounding sentences as metadata
- Pair with `MetadataReplacementNodePostprocessor` — at retrieval time, expands the retrieved sentence back to its full sentence window
- This gives the LLM more coherent, contextually-grounded passages

For structured Sanity content (academy modules, case studies), use `HierarchicalNodeParser` to preserve heading/section structure.

1.5 Add RAG Evaluation and Observability

Current state: There is no system to measure retrieval quality, answer faithfulness, or hallucination rate. Changes to the RAG pipeline are made without knowing if they improve or regress quality.

Problem: Without evaluation, you are flying blind. A chunking change that feels like an improvement may actually reduce faithfulness by 20%.

Recommendation: Integrate a RAG evaluation framework:

- **Ragas** (open source) — measures faithfulness, answer relevancy, context precision, context recall using the LLM itself as evaluator
- **TruLens** — alternative with a nice dashboard UI
- Build a golden test set of 50 question/answer pairs from the knowledge base

- Run evaluation suite on every RAG pipeline change before deploying

Also add observability logging:

- Log every query, retrieved chunks, and response latency to Supabase
- Track which documents are retrieved most/least often (informs content gaps)
- Alert on high latency or zero-result queries

1.6 Persist Conversation Sessions to Database

Current state: Conversation history is stored in `ChatMemoryBuffer` in-process on Railway and in browser `localStorage` on the frontend.

Problem: Railway restarts its service containers on deployment and periodically for maintenance. When this happens, all in-flight conversation context is lost. The user's follow-up question returns to a fresh context. This is a jarring UX failure.

Recommendation: Add a `conversations` table to Supabase:

```
CREATE TABLE conversations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES auth.users NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE conversation_messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  conversation_id UUID REFERENCES conversations NOT NULL,
  role TEXT NOT NULL CHECK (role IN ('user', 'assistant', 'system')),
  content TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
);
```

On each chat request, load prior messages from Supabase and reconstruct the `ChatMemoryBuffer`. On each response, persist the new turn. This also enables:

- "Conversation history" UI showing past sessions
- Analytics on common question patterns
- Fine-tuning dataset collection

1.7 Add Agentic Tool Use

Current state: The AI Analyst is a pure RAG system — it retrieves documents and synthesizes answers. It cannot take actions or query live data.

Problem: Users asking "What is Vermont's current composite score on the HTI?" need the AI to query the `state_health_metrics` table, not retrieve a document that may be months old. Users asking "Run a CEA calculation for this scenario" need the AI to execute a calculator.

Recommendation: Upgrade the AI Analyst to an agentic system with tool use:

- **DatabaseQueryTool** — safely query Supabase state metrics, RHT profiles, hospital data
- **ResearchLabTool** — invoke Research Lab calculators (CEA, APM, HCC scoring) with parameters extracted from the conversation
- **WebSearchTool** — fetch current CMS, KFF, or FDA updates for time-sensitive queries
- **SanitySearchTool** — targeted content retrieval by pillar/type

LlamaIndex has first-class support for `FunctionTool` and `ReActAgent`. This transforms the chatbot from a document reader into a genuine analytical assistant.

1.8 Improve Citation Quality

Current state: The citation button attributes answers to source documents broadly (document title, type).

Problem: For a healthcare policy platform where accuracy is mission-critical, broad citations are insufficient. Users cannot verify specific claims without finding the passage themselves. This creates credibility risk — if an AI answer contains an error, the vague citation makes it harder to detect.

Recommendation:

- Surface specific quoted passages from source documents alongside citations
- Include page number (for PDFs) or section heading (for Sanity content)
- Add a confidence indicator based on cosine similarity score of retrieved chunks
- Add a "Verify this answer" button that shows all retrieved context chunks with their similarity scores

---

1.9 Guard Against System Prompt Leakage

Current state: Tier-aware system prompts differentiate capabilities between Free and Advisory tiers. The prompts are constructed in `main.py` and sent to the LLM with each request.

Problem: A user who intercepts their own streaming response or calls the Railway API directly (bypassing frontend rate limiting) can observe behavioral differences that reveal system prompt content. At the Advisory tier, this could expose proprietary analytical frameworks.

Recommendation:

- Add a layer of indirection — store system prompt templates in Supabase, not in source code
- Never reflect system prompt content in error messages or debug output
- Add a check: if the user asks "What are your instructions?" — respond generically, not with the actual prompt

---

1.10 Add Streaming Resilience

Current state: The `/api/chat` endpoint uses `StreamingResponse` from FastAPI. The frontend receives tokens as they arrive.

Problem: If the Railway service goes cold (Railway free tier hibernates after inactivity), the first request can time out on the frontend before the stream begins. There is no visible feedback during the cold start delay (typically 5–15 seconds).

Recommendation:

- Add a "warming" heartbeat: send a keepalive comment byte every 2 seconds while the LLM is processing, so the frontend connection stays alive
- Add a visible "Analyst is thinking..." indicator that distinguishes cold-start from active processing
- Consider upgrading Railway to a paid tier with always-on service for production

---

2. Architecture

2.1 Refactor Single-File Backend

Current state: The entire FastAPI backend is a single `main.py` file containing routes, middleware, RAG pipeline, ingestion logic, JWT verification, and system prompts.

Problem: This file will grow unmanageable. Adding a new tool, a new auth method, or a new data source requires editing a single shared file with no separation of concerns. Testing individual components is difficult.

Recommendation: Refactor to a standard FastAPI module structure:

```
```

```
backend/
├── main.py              # App factory, startup, middleware registration
├── routers/
│   ├── chat.py          # /api/chat endpoint
│   ├── ingest.py        # /api/ingest endpoint
│   └── suggest.py       # /api/suggest endpoint
├── services/
│   ├── rag.py           # LlamaIndex pipeline, retrieval, generation
│   ├── ingestion.py     # PDF + Sanity ingestion logic
│   └── auth.py          # JWT verification, role extraction
├── models/
│   └── schemas.py       # Pydantic request/response models
├── prompts/
│   └── system_prompts.py # Tier-aware system prompt templates
└── config.py            # Settings via pydantic-settings
```

2.2 Move to Monorepo Tooling

Current state: Frontend (Next.js) and backend (FastAPI) are co-located but managed as completely independent projects with no shared tooling, no shared types, and no unified build pipeline.

Problem: API contract changes (new field, renamed parameter) can break the frontend silently. There is no mechanism to enforce consistency between the TypeScript types on the frontend and the Pydantic schemas on the backend.

Recommendation:

- Add a `vercel.json`-aware Turborepo configuration at the root
- Use `openapi-typescript` to auto-generate TypeScript types from the FastAPI OpenAPI spec — this creates a typed contract between frontend and backend
- Add a root-level `Makefile` or `package.json` with commands for starting both services, running tests, and building

---

2.3 Add Background Job Queue for Ingestion

Current state: The `POST /api/ingest` endpoint is synchronous — it processes all PDFs and fetches all Sanity content in a single HTTP request that must complete before the response is sent.

Problem: As the knowledge base grows (more PDFs, more Sanity content types, more articles), this endpoint will timeout. Railway has a default 30-second request timeout. A full knowledge base rebuild may take 5–10 minutes.

Recommendation:

- Add a simple job queue using Supabase as the backing store (a `ingest_jobs` table with status: queued/running/completed/failed)
- The `/api/ingest` endpoint enqueues a job and returns `202 Accepted` immediately
- A separate worker process (or Railway cron) polls for queued jobs and runs them
- Add a `GET /api/ingest/status` endpoint the frontend can poll to show rebuild progress
- Alternatively, use Railway's built-in cron to trigger scheduled rebuilds overnight

---

2.4 Infrastructure-as-Code

Current state: Infrastructure is configured via Vercel dashboard, Railway web UI, and Supabase web UI. There are no Terraform/Pulumi definitions for any of this.

Problem: If you need to recreate the infrastructure from scratch (disaster recovery, new environment, team member onboarding), there is no reproducible path. Manual configuration drift between environments is invisible.

Recommendation:

- Use Supabase CLI + migration files (partially done) — make these authoritative
- Add Vercel CLI project configuration to version control
- Document Railway environment variables in a `.env.railway.example` file
- Consider Pulumi for managing Supabase, Vercel, and Railway as code (all three have Pulumi providers)

2.5 Add a Staging Environment

Current state: Development runs locally; production is on Vercel + Railway. There is no intermediate environment.

Problem: Changes go directly from a developer's laptop to production users. A broken build, a broken migration, or a misconfigured environment variable affects real paying subscribers.

Recommendation:

- Create a `staging` branch in GitHub
- Configure Vercel preview deployments for the `staging` branch
- Deploy a second Railway service for the staging backend
- Create a `staging` Supabase project (or use Supabase branching, which is now available)
- All PRs should deploy to staging first; promote to production after review

---

3. Database

3.1 Audit and Enforce Row-Level Security

Current state: RLS is referenced in documentation but enforcement across all tables is not confirmed in the migration files.

Problem: If the `rag_documents`, `state_health_metrics`, or `rht_state_profiles` tables are queryable via the `anon` key without RLS, unauthenticated users can access paywalled data by calling the Supabase REST API directly — completely bypassing the frontend.

Recommendation: For every table, run this audit:

```
-- Find tables without RLS enabled
SELECT tablename
FROM pg_tables
WHERE schemaname = 'public'
  AND tablename NOT IN (
    SELECT tablename FROM pg_tables
    WHERE schemaname = 'public'
    AND rowsecurity = TRUE
  );
```

Then add appropriate policies. For subscriber-gated content:

```
ALTER TABLE rag_documents ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Subscribers can read rag_documents"
  ON rag_documents FOR SELECT
  USING (
    EXISTS (
      SELECT 1 FROM user_roles
      WHERE user_id = auth.uid()
      AND role IN ('subscriber','student','professional','advisory','admin')
    )
  );
```

3.2 Add Role Change Audit Log

Current state: The `user_roles` table records a user's current role but not the history of role changes.

Problem: If a user claims they were charged for a plan they didn't receive, or if fraudulent role escalation occurs, there is no audit trail to investigate.

Recommendation:

```
CREATE TABLE role_change_log (
  id BIGSERIAL PRIMARY KEY,
  user_id UUID REFERENCES auth.users NOT NULL,
  old_role TEXT,
  new_role TEXT NOT NULL,
  changed_by UUID REFERENCES auth.users,
  changed_at TIMESTAMPTZ DEFAULT now(),
  reason TEXT  -- 'stripe_webhook', 'admin_override', 'signup_default'
);

-- Trigger to auto-log changes
CREATE OR REPLACE FUNCTION log_role_change()
RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO role_change_log (user_id, old_role, new_role, reason)
  VALUES (NEW.user_id, OLD.role, NEW.role, 'trigger');
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER on_role_change
AFTER UPDATE ON user_roles
FOR EACH ROW EXECUTE FUNCTION log_role_change();
```

3.3 Add Full-Text Search Index

Current state: Search is routed entirely through Sanity's GROQ API. There is no server-side full-text search on the Supabase content tables.

Problem: Sanity GROQ search has latency (~200–400ms round trip). For certain low-latency use cases (command palette autocomplete, live search-as-you-type), this is too slow. Also, content that lives in Supabase tables (state metrics, hospital data) is not searchable at all.

Recommendation: Add PostgreSQL full-text search:

```
-- Add a tsvector column to rag_documents for hybrid search
ALTER TABLE rag_documents ADD COLUMN fts tsvector
  GENERATED ALWAYS AS (to_tsvector('english', coalesce(text, ''))) STORED;

CREATE INDEX rag_documents_fts_idx ON rag_documents USING GIN (fts);

-- Search query
SELECT *, ts_rank(fts, query) AS rank
FROM rag_documents, to_tsquery('english', 'AHEAD & vermont') query
WHERE fts @@ query
ORDER BY rank DESC
LIMIT 10;
```

3.4 Add Metadata Index for Filtered Vector Search

Current state: The `rag_documents` table stores metadata (including `pillar`) as JSON but has no index on that field.

Problem: Filtering vector search by pillar (e.g., "search only Policy documents") requires a full table scan of the `metadata` JSON column, which is slow and gets worse as the knowledge base grows.

Recommendation:

```
-- Add a generated column for pillar
ALTER TABLE rag_documents
  ADD COLUMN pillar TEXT GENERATED ALWAYS AS (metadata->>'pillar') STORED;

CREATE INDEX rag_documents_pillar_idx ON rag_documents (pillar);

-- Now filtered ANN search is efficient:
SELECT * FROM rag_documents
WHERE pillar = 'policy'
ORDER BY embedding <=> '[...]'
LIMIT 10;
```

3.5 Add Conversation History Tables

(See section 1.6 for full schema. This is noted here as a database task.)

3.6 Automate Migrations in CI

Current state: Supabase migration files exist but there is no CI step that validates or applies them.

Problem: A developer writes a migration locally, tests it, but forgets to push it. Production schema diverges from the migration history. Or a migration is pushed that breaks an existing query and only fails at runtime.

Recommendation:

- Add a GitHub Actions step that runs `supabase db push --dry-run` on every PR to validate migration syntax
- Add `supabase db diff` to detect schema drift between local and production
- Tag migrations with a version number and apply them in order on deployment

---

4. APIs & Integrations

4.1 Add API Versioning

Current state: All API routes are unversioned (`/api/chat`, `/api/ingest`, `/api/search`).

Problem: When a breaking change is made to an API contract (adding a required field, changing a response shape), the frontend and backend must be deployed simultaneously. If they are ever out of sync, requests fail silently or with cryptic errors.

Recommendation:

- Version all backend routes: `/api/v1/chat`, `/api/v1/ingest`
- Generate an OpenAPI spec from FastAPI automatically (`fastapi` does this at `/docs`)
- Use `openapi-typescript` to generate TypeScript types from the spec at build time
- Never remove a version without a deprecation period

4.2 Complete Webhook Verification

Current state: Stripe webhooks use signature verification. Other potential webhook sources (Sanity content updates, Resend email events) may not be verified.

Recommendation:

- Audit every `POST` endpoint that receives external data
- Ensure all webhooks are verified with HMAC signatures
- Sanity supports webhook signing — implement it to enable real-time content sync (when a new article is published in Sanity, trigger RAG re-ingestion automatically)

4.3 Upgrade Email to a Proper Platform

Current state: Weekly digest emails are composed programmatically in a Next.js API route and sent via Resend.

Problem: There is no visual template editor, no open-rate or click-rate tracking, no A/B testing capability, and no automated unsubscribe list management (compliance risk).

Recommendation:

- Move to **Loops.so** (built for SaaS, has a visual editor, native Supabase integration) or **Postmark** (transactional email with templates and analytics)
- Build triggered email sequences: welcome series for new subscribers, re-engagement for churned subscribers, weekly digest with open tracking
- CAN-SPAM / CASL compliance: ensure every email has a one-click unsubscribe that immediately updates Supabase

---

4.4 Add Webhook Dead Letter Queue

Current state: Stripe webhook processing logs events to `stripe_events` for idempotency, but if the database is unavailable when a webhook arrives, the event is lost.

Problem: A failed webhook means a subscription state change (upgrade, downgrade, cancellation) is not reflected in the platform. Users may have incorrect access for hours or days.

Recommendation:

- Store all incoming webhooks in a `webhook_inbox` table immediately on receipt (before any processing)
- Process webhooks asynchronously from this inbox
- Mark failed webhooks for retry with exponential backoff
- Alert on webhooks that have failed more than 3 times

4.5 Add Fallback for News Ticker

Current state: The ticker fetches RSS from KFF Health News, FDA, and CMS every 5 minutes.

Problem: If any of these external feeds are unavailable (they occasionally are), the ticker silently fails or shows nothing.

Recommendation:

- Cache the last successful fetch in Supabase or Redis
- Serve cached headlines if the live fetch fails
- Show a subtle "Last updated X minutes ago" timestamp so users know freshness

---

4.6 Consider a Public Data API

The HTI Index, state performance metrics, and RHT profiles are unique data assets.

Recommendation: Offer a developer/researcher API:

- Authenticated with API keys (separate from user sessions)
- Rate-limited by tier
- Returns state metrics, HTI scores, RHT program details as structured JSON
- Opens a new revenue stream (API tier pricing)
- Generates academic citations and PR when researchers use the data

5. Frontend / UI / UX

5.1 Upgrade React Version

Current state: `react: "19.0.0-rc-66855b96-20241106"` — a specific release candidate build from November 2024.

Problem: RC builds are not stable and may contain bugs that are fixed in the final release. React 19 stable was released in December 2024.

Recommendation: Update to `react: "^19.0.0"` and `react-dom: "^19.0.0"` in `package.json` and test for any breaking changes.

5.2 Add Skeleton Loading States

Current state: Data-heavy pages (dashboard maps, state metrics, HTI timeseries) fetch data on load with no visual placeholder.

Problem: Users see blank space or layout shift while data loads. This creates a perception of slowness and can cause content reflow (poor CLS score for Core Web Vitals).

Recommendation:

- Add Tailwind-based skeleton loaders for all data-fetching components
- The pattern is simple: a div with `animate-pulse bg-gray-200 rounded` that mirrors the shape of the loaded content
- Prioritize: national map, state dashboard cards, HTI chart, article feeds

---

5.3 Add Comprehensive Error Boundaries

Current state: There is no evidence of React `ErrorBoundary` components in the codebase.

Problem: An uncaught JavaScript error in one panel (news ticker, right sidebar chat, D3 map) can crash the entire page. Users see a blank white screen with no explanation.

Recommendation:

- Wrap every independent section in an `ErrorBoundary` component
- Show a graceful fallback: "This section is temporarily unavailable. Refresh to retry."
- Log boundary-caught errors to Sentry (see section 7.1)
- The AI chat widget and the map visualizations are highest priority for this

---

5.4 Extend the Command Palette

Current state: The ⌘K command palette searches content (articles, modules, glossary) but cannot navigate to routes or launch tools.

Problem: Power users expect ⌘K to be a full keyboard-driven control center. Limiting it to content search undersells the feature.

Recommendation: Add these command categories to the palette:

- **Navigate to**: Go to Research Lab, Advisory, Dashboard, Chat, Academy, Settings
- **Launch tool**: Open APM Calculator, CEA Calculator, FHIR Lab, HCC Scoring
- **Action**: Start new conversation, Upgrade plan, Download PDF, Share current page
- **Content search**: (current functionality, keep it)

5.5 Add Dark Mode

Current state: White/light theme only.

Problem: Healthcare professionals frequently use digital tools in dim environments (clinical settings, late-night policy review). Dark mode is a standard expectation for any professional SaaS platform in 2025+.

Recommendation:

- Add dark mode using Tailwind's `dark:` prefix and CSS variables for colors
- Toggle via a sun/moon icon in the header (respect `prefers-color-scheme` as default)
- Persist preference in `localStorage` and user profile settings

5.6 Fix Mobile Experience

Current state: The platform's sidebar layout, research lab tools, and D3 dashboard maps are designed for desktop and likely render poorly on mobile screens.

Problem: A hospital executive reviewing state performance data on their phone has a poor experience. The AI chat widget on mobile may be unusable.

Recommendation:

- Run a mobile audit of the top 10 most-used pages using Chrome DevTools device emulation
- Prioritize: homepage, pillar article pages, AI chat, and state dashboard
- Research Lab tools: on mobile, show a "best experienced on desktop" notice rather than a broken layout
- Consider a dedicated mobile-optimized "quick brief" view for each pillar

---

5.7 Add Interactive Onboarding Tour

Current state: New users land on a feature-rich platform with no guided introduction.

Problem: The platform has 19 research tools, 5 content pillars, an AI analyst, state dashboards, an academy, and an advisory hub. First-time users do not know where to start and are likely to churn before discovering the features most relevant to them.

Recommendation:

- Add a role-based onboarding tour (using `driver.js` or `Shepherd.js`) triggered on first login
- Ask 3 questions during onboarding: "What is your role?" (hospital exec, state official, researcher, consultant), "What is your primary interest?" (policy, economics, clinical, etc.), "What is your goal?" (stay current, do analysis, learn, advise clients)
- Route new users to the most relevant starting point based on answers
- Highlight: the AI Analyst, the most relevant pillar, and the Research Lab tool most relevant to their role

---

5.8 Consolidate Styling Systems

Current state: The application uses both `styled-components` and Tailwind CSS v4 (beta).

Problem: Two styling systems create redundancy — two ways to write CSS, two sets of bundle overhead, inconsistent patterns across components, and slower developer onboarding.

Recommendation:

- Commit to Tailwind CSS as the primary styling system
- Progressively migrate `styled-components` usage to Tailwind utilities
- The `styledComponents: true` compiler flag in `next.config.ts` can remain during migration
- Also: upgrade Tailwind from v4 beta to v4 stable when released

---

5.9 Add Content Action Improvements

Current state: Articles have share, print, and save-to-PDF buttons.

Recommendation — Additional content actions:

- **Bookmark**: Save articles to a personal reading list (store in Supabase per user)
- **Highlight & Annotate**: Let subscribers highlight passages and add private notes (like Substack's notes feature — valuable for policy researchers)
- **Email to colleague**: One-click to share a direct article link via email
- **Export to Notion/Google Docs**: For Professional+ users who use these tools for research synthesis

---

5.10 Improve Empty States

When a user visits a page with no content loaded (e.g., empty Research Workspace, no saved courses, no conversation history), they currently see blank space.

Recommendation: Design purposeful empty states for every list/collection page:

- Illustration + explanation of what the section is for
- A clear CTA: "Start your first research session", "Enroll in a course", "Ask the Analyst"
- Empty states are a conversion opportunity, not just a UI detail

---

6. Security

6.1 Audit Supabase Key Exposure

Current state: `NEXT_PUBLIC_SUPABASE_ANON_KEY` is exposed to the browser (by design — it's meant to be public). `SUPABASE_SERVICE_ROLE_KEY` must never appear in client-side code.

Recommendation:

- Grep the entire `frontend/` directory for `SERVICE_ROLE_KEY` — ensure it never appears in any file that is not a server-side API route
- The anon key is safe only if RLS is properly enforced on every table (see section 3.1)
- Add a comment in each API route that uses the service role key explaining why it needs elevated access

---

6.2 Add Rate Limiting to FastAPI Backend

Current state: Rate limiting exists only on the Next.js frontend. The FastAPI backend on Railway has no rate limiting.

Problem: Anyone who discovers the Railway backend URL (it's not secret — it's in the `vercel.json` rewrite) can call `/api/chat` directly, bypassing the frontend rate limiter. This can exhaust Groq API credits or cause a DoS.

Recommendation: Add `slowapi` to the FastAPI backend:

```
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

@app.post("/api/chat")
@limiter.limit("20/minute")
async def chat(request: Request, ...):
    ...
```

6.3 Move Research Lab Business Logic Server-Side

Current state: All 19 Research Lab tool computations run entirely in browser JavaScript.

Problem: The actuarial models, HCC v28 scoring, APM design calculations, and policy simulation logic are fully visible and reversible-engineered by any competitor in minutes. These models are a core intellectual property asset of the platform.

Recommendation: Move the most proprietary calculations to server-side API endpoints:

- HCC Risk Stratification Engine
- Actuarial Lab
- APM Design Lab
- CEA Calculator (condition-specific parameters)

Simpler UI tools (sliders, formatters) can remain client-side. The core IP should not be.

6.4 Add PHI Detection to Chat Input

Current state: The AI chat accepts any text input from users. The Terms of Service presumably prohibit PHI input, but there is no technical enforcement.

Problem: If a user pastes patient data into the chat (e.g., "my patient John Smith DOB 01/01/1980 has these claims..."), this PHI is sent to Groq's API and may be stored in conversation logs. This creates HIPAA exposure.

Recommendation:

- Add a client-side PHI detection layer before submitting messages
- Use regex patterns for common PHI: SSN, MRN formats, date-of-birth with name proximity
- If PHI is detected, show a warning: "This message may contain patient information. Remove it before sending — the platform does not support PHI."
- Add this explicitly to the Terms of Service with a conspicuous notice in the chat UI

---

7. Performance & Observability

7.1 Add Application Performance Monitoring

Current state: No Sentry, Datadog, New Relic, or equivalent APM is integrated.

Problem: Errors in production are invisible until a user reports them. There is no way to know if a deployment broke a feature for a segment of users, or if a particular page is crashing for certain browsers.

Recommendation:

- Add **Sentry** to both the Next.js frontend and the FastAPI backend (Sentry has SDKs for both)
- Frontend: captures JavaScript errors, React component stack traces, and page performance
- Backend: captures Python exceptions with full traceback and request context
- Configure performance monitoring to track p50/p95/p99 latency for chat and search endpoints
- Set up Slack alerts for new high-frequency errors

7.2 Add Caching Layer

Current state: State metrics, RHT profiles, news ticker, and Sanity content are fetched fresh on each request.

Problem: `state_health_metrics` does not change more than once per day. Fetching it fresh on every page load is wasteful and adds latency. The Sanity GROQ API is called for every search and article render.

Recommendation:

- Add **Upstash Redis** (serverless, works well with Vercel and Railway) as a caching layer
- Cache state metrics for 1 hour (TTL: 3600)
- Cache Sanity article content for 15 minutes (TTL: 900) — or use Next.js `revalidate` with ISR (Incremental Static Regeneration)

- Cache news ticker results for 5 minutes (TTL: 300) — already refreshes every 5 min, so cache aligns perfectly
- Use Next.js `unstable_cache` for server component data fetching

7.3 Establish Core Web Vitals Baseline

Current state: No Lighthouse or Web Vitals tracking has been established.

Problem: With D3 maps, complex sidebar layouts, and AI streaming, LCP (Largest Contentful Paint), CLS (Cumulative Layout Shift), and INP (Interaction to Next Paint) scores are likely poor. Poor CWV hurts SEO ranking and user experience simultaneously.

Recommendation:

- Run Lighthouse audit on the top 5 pages, record baseline scores
- Add `next/web-vitals` reporting to track real-user CWV
- Target: LCP < 2.5s, CLS < 0.1, INP < 200ms
- Quick wins: add `width` and `height` to all `<Image>` components, defer D3 map initialization until after first paint, lazy-load the right sidebar chat widget

---

7.4 Optimize D3 Map Performance

Current state: The national map uses D3/react-simple-maps with a US Atlas TopoJSON file loaded from jsDelivr CDN.

Problem: The TopoJSON file is ~200KB. Loading it synchronously blocks first paint. The D3 rendering is CPU-intensive and causes layout jank on lower-end devices.

Recommendation:

- Download the TopoJSON file and serve it from `/public` (removes CDN dependency, also eliminates the jsDelivr entry in the CSP `connect-src` directive)
- Lazy-load the map component: `const NationalMap = dynamic(() => import('./NationalMap'), { ssr: false })`
- Add a skeleton placeholder while the map loads
- Consider pre-rendering a static SVG snapshot of the map for initial paint, then hydrating with interactive D3 after load

---

8. Testing & DevOps

8.1 Add an Automated Test Suite

Current state: `TESTING_AND_DEPLOYMENT_GUIDE.md` documents a testing approach, but there are no actual test files in the codebase.

Problem: Every code change is deployed untested. A regression in auth, subscription gating, or the Stripe webhook handler is discovered only by a paying user encountering it in production.

Recommendation — Minimum viable test suite:

Frontend (Jest + React Testing Library):

```
frontend/__tests__/
├── auth/
│   ├── login.test.tsx          # Login form validation and submission
│   └── signup.test.tsx         # Signup flow with role assignment
├── subscription/
│   ├── gating.test.tsx         # Content gating by role
│   └── stripe-webhook.test.ts  # Webhook event handling
├── chat/
```

```
│   └── chat-api.test.ts        # Chat API proxy and streaming
└── utils/
    └── formatters.test.ts      # Currency, date, percentage formatters
```

Backend (Pytest):

```
backend/tests/
├── test_auth.py               # JWT verification, role extraction
├── test_chat.py               # Chat endpoint, tier-aware prompts
├── test_ingest.py             # PDF + Sanity ingestion pipeline
└── test_suggest.py            # Follow-up suggestion generation
```

E2E (Playwright):

```
e2e/
├── auth-flow.spec.ts           # Signup → verify → onboard → login
├── subscription-upgrade.spec.ts # Free → Subscriber via Stripe
├── chat-conversation.spec.ts   # Subscriber asks a question, gets a response
└── research-lab.spec.ts        # Open APM calculator, enter values, get output
```

8.2 Add CI/CD Pipeline

Current state: No GitHub Actions workflows exist. Deployments are manual.

Problem: Code can be merged and deployed with no automated quality checks. A TypeScript error, a broken build, or a failing test can reach production.

Recommendation: Add these GitHub Actions workflows:

`.github/workflows/ci.yml` (runs on every PR):

```yaml
jobs:
  frontend-checks:
    - npm install
    - npx tsc --noEmit          # Type check
    - npm run lint               # ESLint
    - npm run build              # Build check
    - npm test                   # Jest tests

  backend-checks:
    - pip install -r requirements.txt
    - mypy main.py              # Type check
    - pytest tests/             # Unit tests
    - ruff check .              # Linting

  e2e-tests:
    - Run Playwright against staging
```

`.github/workflows/deploy.yml` (runs on merge to `main`):

```yaml
jobs:
  deploy-backend:
    - railway up --service backend

  deploy-frontend:
    - vercel --prod
```

8.3 Add Dependency Update Automation

Current state: Dependencies are pinned but manually managed.

Problem: Security vulnerabilities in `next`, `stripe`, `llama-index`, or `fastapi` will not be automatically surfaced.

Recommendation:

- Enable **Dependabot** in the GitHub repository (add `.github/dependabot.yml`)
- Configure weekly dependency update PRs for both `frontend/` and `backend/`
- Review and merge security-critical updates immediately

---

9. Domain Knowledge & Content Depth

9.1 Expand Beyond Vermont-Centric Coverage

Current state: Deep Vermont-specific content exists (Act 167, AHEAD Model, NVRH hospital data) but national coverage is thin — primarily RHT state profiles and aggregate metrics.

Problem: The platform markets itself as national healthcare transformation intelligence, but a subscriber in North Carolina, Texas, or California has little state-specific content to justify their subscription.

Recommendation:

- Commit to 3–5 deep-dive state analyses per quarter, rotating through states with major transformation initiatives
- Priority states: California (CalAIM/Medi-Cal, started but incomplete), North Carolina (statewide transformation program), Massachusetts (AHEAD Model analog), Texas (Medicaid managed care scale)
- Create a "State Intelligence Series" as a branded content track within the Academy

---

9.2 Deepen the Clinical Pillar

Current state: The Clinical pillar (Hospital-at-Home, precision medicine, virtual care, population health) has fewer articles and less analytical depth than Policy, Economics, and Technology.

Problem: Clinical leaders (CMOs, CNOs, physician executives) are a high-value subscriber segment. The current clinical content does not justify their subscription the way the policy and economics content justifies a policy analyst's subscription.

Recommendation:

- Add dedicated clinical content: workforce burnout analytics, nurse staffing ratio impacts, remote patient monitoring ROI, hospital-at-home program case studies
- Partner with a clinical faculty member or CMO-level advisor to generate and validate content
- Add clinical CE/CME certification tracks in the Academy

9.3 Publish Original Data and Research

Current state: All content analyzes secondary sources. The HTI Index is the only original data asset.

Problem: Secondary analysis is abundant. Original data is scarce and valuable. Platforms that produce original data get cited, linked to, and talked about in the industry. This is the highest-ROI content investment for SEO, PR, and subscriber acquisition.

Recommendation:

- Annual "State of Healthcare Transformation" survey (500+ respondents: hospital execs, state officials, consultants) — publish findings as a flagship report

- Expand the HTI methodology to include leading indicators (legislation introduced, pilots launched) not just lagging outcomes
- Partner with a university health policy department to co-publish findings (adds academic credibility and distribution)

9.4 Connect the Glossary to Article Content

Current state: The glossary exists as a Sanity schema with healthcare terminology definitions, but it appears disconnected from article content.

Problem: A non-expert reader encountering "AHEAD Model," "risk stratification," or "global budget" in an article has no way to get an inline definition without leaving the page.

Recommendation:

- Implement inline term definitions: when a glossary term appears in article body text, automatically link it to a hover tooltip showing the definition
- This is a Sanity Portable Text customization — add a custom annotation type for `glossaryTerm` references
- This dramatically improves comprehension for new subscribers and reduces the expertise barrier to entry

---

9.5 Add Analyst Credentials and Methodology Transparency

Current state: Team profiles (Dr. Sarah Chen, Marcus Webb, Dr. Priya Nair) are referenced in content but there are no public-facing author pages with credentials, publications, or areas of expertise.

Problem: Healthcare policy professionals are skeptical. They need to know who is doing the analysis. "Intellectual Rigor" is a stated value — but without named, credentialed analysts, it is an assertion, not a proof.

Recommendation:

- Publish full analyst bios with academic credentials, prior roles, and publication records
- Add author pages that list all content published by each analyst
- Publish methodology notes for the HTI Index, the Research Lab tools, and the AI Analyst (what data sources, what assumptions, what limitations)
- Consider an annual "Methodology Update" document that details any changes to scoring and why

---

10. Mission / Vision / Values / About

10.1 Make Values Operational, Not Aspirational

Current state: The six core values (Intellectual Rigor, Editorial Independence, Systemic Thinking, Community First, Cross-Disciplinary Collaboration, Radical Transparency) read as principles without operational commitments.

Problem: Values that exist only as statements are marketing copy. Values that include specific behavioral commitments are trust signals. For a healthcare intelligence platform, trust is the product.

Recommendation: Add a specific commitment beneath each value:

Value	Aspirational Statement	Operational Commitment
Intellectual Rigor	Grounded in primary sources	"We cite every factual claim. See our sourcing standards."
Editorial Independence	No sponsored content	"We have never accepted sponsored content. Here is our funding disclosure."
Radical Transparency	Publish methodology	"Read our HTI methodology. See our revision history."

Value	Aspirational Statement	Operational Commitment
Community First	Accountable to underserved	"X% of our research hours are dedicated to equity analysis."

10.2 Rewrite the Mission Statement for External Audiences

Current state: Mission: "Provide intelligence infrastructure for healthcare leaders navigating structural transformation."

Problem: This is internally coherent but externally abstract. A prospective subscriber landing on the `/mission` page does not immediately understand what the platform *does differently* or *why they should care*.

Recommendation — Draft alternatives to workshop:

Option A (Outcome-focused):

> "We give healthcare leaders the analysis they need to make transformation decisions with confidence — not guesswork."

Option B (Problem-focused):

> "Healthcare transformation is fragmented, politicized, and poorly documented. We cut through the noise with rigorous, independent analysis across policy, economics, technology, clinical practice, and equity."

Option C (Evidence-focused):

> ☑ analyses reviewed by [Y] healthcare leaders in [Z] states. We are the only platform that tracks all five dimensions of transformation simultaneously."

Whichever direction you choose, pair the mission statement with 3–5 concrete proof points: dollar value of contracts analyzed, states covered, research hours published, subscriber roles.

10.3 Put Real Faces on the Team

Current state: Team profiles (Dr. Sarah Chen, Marcus Webb, Dr. Priya Nair) are referenced in content code, but if these are placeholder personas, there are no real faces on the brand.

Problem: Anonymous expertise does not build trust in healthcare. Every credible healthcare analytics platform (Advisory Board, Chartis, Sg2) leads with named, credentialed people.

Recommendation:

- If team members are real, add professional headshots, LinkedIn links, and publication records
- If team is small, that is fine — authenticity beats inflated team pages
- Consider adding an Advisory Board section: 3–5 named healthcare leaders who can lend credibility and open doors to institutional subscribers

---

10.4 Add a Compelling Origin Story

Current state: The `/mission` page has a milestones timeline (2019–2026) listing *what* happened but not *why* the platform was founded.

Problem: Healthcare SaaS platforms with a compelling founding story have substantially higher conversion on mission/about pages. People connect with problems that were lived, not described.

Recommendation: Add a "Why We Started" section:

- What problem did the founder(s) experience firsthand?
- What was the gap in the market?
- What was the insight that no one else had seen?

- Why healthcare, why transformation, why this five-pillar framework?

10.5 Add a "Who We Serve" Section

Current state: The About page describes what the platform does but does not explicitly describe who it is for.

Recommendation: Add an explicit audience section with 4–6 subscriber personas:

- **State Medicaid Director**: "You manage a $3B program. We give you the policy precedent and economic modeling to justify your decisions to the legislature."
- **Hospital CFO**: "You are evaluating a move to value-based contracts. Our Research Lab runs the financial models. Our case studies show you who succeeded and why."
- **Health Policy Researcher**: "You need current data across 50 states. Our HTI Index and state dashboards give you a clean dataset with documented methodology."
- **Healthcare Consultant**: "Your clients need credible analysis. Our Advisory tier gives you access to our analyst team and proprietary research."

---

11. Marketing & Growth

11.1 Build a Public SEO Content Layer

Current state: All content is gated behind authentication. Non-authenticated visitors cannot see any analysis, which means search engines cannot index it either.

Problem: This is a zero-SEO strategy. The platform produces highly specialized content for terms with commercial intent ("Vermont AHEAD Model analysis," "value-based care state comparison," "HCC risk stratification guide") but captures none of that organic traffic.

Recommendation:

- Create a "Perspectives" or "Briefings" section with ungated thought leadership content
- Publish 2–4 ungated pieces per month: policy briefings, state spotlights, methodology explainers, tool guides
- Add proper SEO metadata to all pages: Open Graph, Twitter Card, `schema.org/MedicalWebPage` structured data, article sitemaps
- Build internal linking between ungated content → upgrade CTA → subscription

---

11.2 Add Article Preview for Non-Members

Current state: Non-authenticated visitors see a hard paywall on all analysis content.

Problem: A visitor who finds an article via Google or a shared link sees a paywall immediately. They have zero context for the value of the content. Conversion rate from hard paywalls is very low.

Recommendation:

- Show the first 2–3 paragraphs of every article before the paywall
- Add a specific, content-aware upgrade CTA: "This analysis continues with [key finding]. Subscribe to read the full analysis — $29/month."
- The teaser should be the most compelling part of the article — write it as a hook

---

11.3 Add Social Proof to Pricing Page

Current state: The pricing page lists plans and features but has no testimonials, case studies, or organizational logos.

Problem: A healthcare professional evaluating a $99/month subscription needs validation that peers like them find value in the platform.

Recommendation:

- Add 3–5 testimonials from real subscribers with name, role, and organization
- If early-stage and testimonials aren't available, use "used by teams at [Organization]" with logos (even 2–3 real logos are powerful)
- ☑ healthcare leaders who read the HTR weekly"

11.4 Add Multiple Email Capture Points

Current state: Email capture is available at `/subscribe` but is not surfaced elsewhere.

Problem: The vast majority of visitors leave without subscribing. Once they're gone, there is no way to re-engage them. Email capture is the highest-ROI retention mechanism for content platforms.

Recommendation:

- Add a persistent "Free weekly briefing" opt-in banner in the site header
- Add an inline email capture at the end of every ungated article
- Add a contextual exit-intent popup for visitors who have scrolled 50%+ of an article
- Add email capture in the footer (standard placement, expected by users)
- Segment captures: "Policy briefing," "Economics briefing," or "Full HTR digest" based on the content the visitor was reading

---

11.5 Add a Referral Program

Current state: No referral mechanism exists.

Problem: The ideal acquisition channel for a healthcare B2B platform is peer recommendation — a state Medicaid director tells a peer at another state, a consultant recommends it to a client. There is no incentive structure to encourage this.

Recommendation:

- Add a referral program: "Share your referral link. Each subscriber who signs up gives you 1 month free."
- Display referral links in the `/account` dashboard
- Send referral stats in the weekly digest ("Your referrals this month: X")
- This is particularly powerful in tight professional communities like state health officials

---

11.6 Implement a Full SEO Strategy

Current state: No structured SEO metadata, no article sitemaps, no schema.org markup, no Open Graph tags.

Recommendation — Quick wins:

- Add `<title>` and `<meta description>` to every page (many Next.js pages likely missing this)
- Add Open Graph and Twitter Card metadata for all article pages (enables rich previews when shared on LinkedIn, which is primary distribution for healthcare policy content)
- Add `schema.org/Article` JSON-LD to analysis pages
- Add a dynamic XML sitemap for all articles, modules, and case studies
- Submit sitemap to Google Search Console

---

12. Business Model & Pricing

12.1 Fix Student vs. Subscriber Pricing Inversion

Current state: Student ($49/month) is priced higher than Subscriber ($29/month) despite students being a price-sensitive demographic.

Problem: This is almost certainly suppressing student adoption, which is a powerful long-term funnel. Today's MPH student or health policy PhD candidate is tomorrow's state Medicaid director or hospital CMO. Acquiring them at low cost now creates a decade-long subscriber relationship.

Recommendation:

- Price Student at $19/month (below Subscriber — justified by educational mission)
- Require `.edu` email verification for Student tier
- Add institution-level verification for university programs
- Consider a "Classroom" plan: one instructor license grants 20–30 student access for a course, billed to the institution

---

12.2 Add Annual Billing Option

Current state: All plans are monthly billing only.

Problem: Monthly-only billing is leaving significant revenue and retention on the table. Annual plans reduce churn dramatically (annual subscribers are 3–5x less likely to cancel in any given month). Annual upfront payment also improves cash flow.

Recommendation:

- Add annual pricing at 20% discount for all tiers:
  - Subscriber: $290/year (save $58 vs. monthly)
  - Student: $190/year
  - Professional: $990/year (save $198)
- Display both monthly and annual on the pricing page with "Best Value" badge on annual
- Default the toggle to annual billing

12.3 Add a Team / Organization Plan

Current state: The jump from Professional ($99/month, 1 user) to Advisory (custom enterprise) is a large cliff with nothing in between for small teams.

Problem: A 3-person consulting firm, a state agency team, or a small hospital system analytics team all need multi-seat access. They are currently forced to either buy 3 individual Professional plans (expensive, unmanaged) or escalate to Advisory pricing (more than they need).

Recommendation: Add a "Team" plan:

- 3–10 seats
- Price: $249/month for 3 seats ($83/user), volume discounts at 5 and 10
- Includes: Professional-tier access for all seats + team admin dashboard + usage reporting
- Self-serve seat management in `/account`

12.4 Improve Freemium Conversion Funnel

Current state: "Free" users can read public articles. The path to upgrade is not contextually triggered.

Problem: Freemium conversion requires showing the value of paid features at the exact moment the user wants them — not on a generic pricing page.

Recommendation — Add contextual upgrade triggers:

- When a free user tries to open AI Analyst → "The AI Analyst is available to Subscribers. Upgrade to get instant answers to your policy questions. First 7 days free."
- When a free user clicks a state dashboard → "Detailed state performance data is available to Subscribers. Here's a preview of Vermont's scores..."
- When a free user tries a Research Lab tool → "Run unlimited analyses in the Research Lab. Upgrade to Professional."
- Each CTA should be specific to the feature being blocked — not a generic "upgrade"

12.5 Add a Free Trial

Current state: The pricing model goes from Free (limited) directly to paid subscriptions with no trial mechanism.

Problem: A $29/month subscription requires trust. A 7-day or 14-day free trial of the full Subscriber plan allows users to experience the AI Analyst, state dashboards, and full article library before committing.

Recommendation:

- Add a "7 days free" trial for Subscriber and Professional plans
- Stripe supports this natively (`trial_period_days` in the subscription creation)
- Require credit card at signup (higher intent than no-card trials)
- Send a "3 days left in your trial" reminder email

---

13. Community & Ecosystem

13.1 Add a Discussion Layer

Current state: Users read analysis in isolation. There is no way for subscribers to discuss content with each other or with the analysts.

Problem: The most valuable insight often comes from how practitioners interpret and apply analysis to their specific context. A hospital CFO reading a VBC analysis wants to know how peers at other hospital systems are thinking about it. This social layer is absent.

Recommendation — Options by complexity:

- **Low effort**: Add comments to articles (Supabase-backed, subscriber-only)
- **Medium effort**: Add a discussion tab per article with threaded comments
- **High effort**: Build a standalone community forum (or use Discourse/Circle.so)
- **Minimum viable**: A subscriber-only Slack workspace where analysts share weekly insights and subscribers can ask questions

---

13.2 Add Live Events and Office Hours

Current state: The Academy has a webinars schema but there is no visible calendar of upcoming events.

Problem: Live events create urgency and community. A subscriber who attends an analyst office hour every month has vastly higher retention than one who only reads asynchronously.

Recommendation:

- Monthly "Policy Briefing" webinars: 30-minute analyst presentation on the most significant healthcare policy development of the month, with Q&A
- Quarterly "State of Transformation" virtual summit: 2-hour event with multiple analysts and guest speakers from state agencies or hospital systems
- Advisory-tier "Strategy Sessions": monthly small-group calls with senior analysts

---

13.3 Add Benchmarking and Peer Comparison

Current state: Research Lab tools run calculations in isolation with no comparison context.

Problem: Knowing that your hospital's VBP adoption rate is 34% is only useful if you know the benchmark. Is 34% good for a critical access hospital in a rural state? Bad? Average?

Recommendation:

- Aggregate anonymized Research Lab results across platform users to build benchmarks
- Show percentile rankings: "Your shared savings rate is in the 67th percentile for similar-sized ACOs in the Northeast"
- This requires users to opt into benchmarking (offer it as a feature, not a default)
- The benchmark data itself becomes a proprietary asset — another reason to subscribe

---

13.4 Build Partner and Integration Ecosystem

Current state: The platform is a standalone product with no integration partnerships.

Recommendation:

- **EHR integrations**: Partner with Epic/Cerner to surface HTR policy alerts within clinical workflows (advanced, but high value for hospital subscribers)
- **State agency partnerships**: Offer a "Government" pricing tier and partner with state health departments to use HTR data in their transformation planning
- **Academic partnerships**: Partner with 2–3 university health policy programs to offer platform access as part of their curriculum (drives student adoption)
- **Consultant partnerships**: Create a "Partner" program for consulting firms who recommend HTR to clients (revenue share or co-branding)

---

14. Accessibility & Compliance

14.1 WCAG 2.1 AA Compliance

Current state: No formal accessibility audit has been performed.

Problem: Many potential institutional customers (state agencies, hospital systems, university programs) require WCAG 2.1 AA compliance in vendor agreements. Without it, the platform cannot be sold to a significant segment of the most valuable customers.

Recommendation:

- Run an automated audit with `axe-core` or Lighthouse Accessibility (identify low-hanging fruit: missing alt text, insufficient color contrast, missing form labels)
- Address critical issues: keyboard navigation for all interactive elements, screen reader compatibility for D3 maps (add ARIA labels and tabular fallbacks), focus indicators
- Target WCAG 2.1 AA compliance within 2 quarters
- Add an Accessibility Statement to the site footer

14.2 CAN-SPAM / CASL Email Compliance

Current state: The weekly digest has an unsubscribe link, but the completeness of compliance is unclear.

Problem: Non-compliance with CAN-SPAM (US) or CASL (Canada — relevant given Vermont's border) can result in fines and reputational damage.

Recommendation:

- Every marketing email must include: physical mailing address, unsubscribe link, clear sender identification, and non-deceptive subject lines
- Unsubscribe requests must be honored within 10 business days (CAN-SPAM) — ensure the Resend unsubscribe webhook updates Supabase immediately
- Maintain a suppression list of unsubscribed addresses

14.3 HIPAA Considerations

Current state: The platform analyzes health policy and aggregate data, not individual patient data. However, the AI chat could receive PHI from users.

Recommendation:

- Add a PHI detection layer (see section 6.4)
- Add conspicuous notice in the chat UI: "Do not enter patient information"
- Update Terms of Service with an explicit PHI prohibition

- If any future features involve individual patient data (e.g., personalized risk scoring), a formal HIPAA compliance program and BAA with cloud vendors will be required

---

14.4 Add Spanish Language Support

Current state: The platform is English-only.

Problem: Vermont has a significant French-speaking population (border communities) and the equity mission of the platform requires reaching health leaders serving non-English-speaking communities. At the national level, Spanish is essential for equity-focused content.

Recommendation:

- Start with Spanish translations of the most-accessed equity pillar content
- Add a language toggle in the header
- Use Next.js internationalization (`i18n` routing) for scalable multi-language support
- Longer term: French for Vermont/New England audience

---

15. Priority Matrix

Priority	Improvement	Impact	Effort	Section
P0	RAG hybrid search (BM25 + vector)	Very High	Medium	1.2
P0	RAG re-ranking step	Very High	Low	1.3
P0	Fix semantic chunking	Very High	Low	1.4
P0	Annual billing option	High	Low	12.2
P0	Student pricing fix ($19, not $49)	High	Low	12.1
P0	Sentry error monitoring	High	Low	7.1
P0	RLS audit and enforcement	High	Low	3.1
P0	Real team profiles with credentials	Very High	Low	9.5, 10.3
P1	RAG evaluation framework (Ragas)	High	Medium	1.5
P1	Persist conversations to Supabase	High	Medium	1.6
P1	FastAPI rate limiting	High	Low	6.2
P1	Upgrade React 19 RC to stable	Medium	Low	5.1
P1	Public ungated SEO content layer	High	Medium	11.1
P1	Article preview for non-members	High	Low	11.2
P1	7-day free trial	High	Low	12.5
P1	CI/CD pipeline (GitHub Actions)	High	Medium	8.2
P1	Skeleton loading states	Medium	Low	5.2
P1	Error boundaries	High	Low	5.3
P1	Contextual upgrade prompts	High	Medium	12.4
P2	Agentic tool use in chatbot	Very High	High	1.7
P2	Onboarding tour	Medium	Medium	5.7
P2	Team/organization plan	High	Medium	12.3
P2	Role change audit log	Medium	Low	3.2
P2	Model routing by tier	Medium	Medium	1.1

Priority	Improvement	Impact	Effort	Section
P2	Refactor backend to modules	Medium	Medium	2.1
P2	Social proof on pricing page	High	Low	11.3
P2	Referral program	High	Medium	11.5
P2	Dark mode	Medium	Medium	5.5
P2	Email platform upgrade (Loops)	Medium	Medium	4.3
P2	Mobile layout audit	Medium	High	5.6
P3	Benchmarking / peer comparison	High	High	13.3
P3	Community forum / Slack	High	High	13.1
P3	WCAG 2.1 AA compliance	High	High	14.1
P3	Original data / annual survey	Very High	High	9.3
P3	Developer API tier	High	High	4.6

---

16. DEPLOYMENT: Vercel + Railway HOW-TO

Complete Step-by-Step Deployment Guide

This section walks you through deploying the Vermont Health Platform from your local machine to production. The frontend (Next.js) goes to **Vercel**; the backend (FastAPI) goes to **Railway**. Your code is already committed to GitHub — this guide picks up from there.

Overview of What Gets Deployed Where

```
GitHub Repository
       │
       ├── frontend/   ──────→  Vercel  (Next.js, CDN, serverless functions)
       │
       └── backend/    ──────→  Railway (Python FastAPI, always-on server)
```

PART A: Deploy the Backend to Railway

The backend must be deployed first because the Vercel frontend needs the Railway URL to configure its API proxy rewrite.

Step A1: Create a Railway Account

1. Go to https://railway.app
2. Click **"Login"** → **"Login with GitHub"**
3. Authorize Railway to access your GitHub account
4. You will land on the Railway dashboard

Step A2: Create a New Railway Project

1. Click **"New Project"**
2. Select **"Deploy from GitHub repo"**
3. If prompted, click **"Configure GitHub App"** and grant Railway access to your repository
4. Find and select your `Vermont-Health-Platform` repository

5. Railway will detect the repository

Step A3: Configure the Backend Service

Railway needs to know this is a Python/FastAPI app and where to find it.

1. After creating the project, click on the service that was created
2. Click the **"Settings"** tab
3. Under **"Source"**:
   - **Root Directory**: `backend`
   - Railway will now look for `requirements.txt` inside `backend/`
4. Under **"Deploy"**:
   - **Start Command**: `uvicorn main:app --host 0.0.0.0 --port $PORT`
   - (Railway injects the `$PORT` environment variable automatically)
5. Click **"Save"**

> **Note:** The `railway.toml` file in the project root should already configure some of this. Verify Railway picked it up correctly under Settings.

Step A4: Add Backend Environment Variables

1. Click the **"Variables"** tab on your Railway service
2. Click **"Add Variable"** for each of the following:

```
GROQ_API_KEY              = your-groq-api-key
GROQ_MODEL                = llama-3.3-70b-versatile
OPENAI_API_KEY            = your-openai-api-key
SANITY_PROJECT_ID         = fxz10xl7
SANITY_DATASET            = production
SANITY_API_TOKEN          = your-sanity-token
SUPABASE_URL              = https://clryhwqaqhvdikgesjbc.supabase.co
SUPABASE_SERVICE_ROLE_KEY = your-supabase-service-role-key
SUPABASE_JWT_SECRET       = your-supabase-jwt-secret
SUPABASE_DB_URL           = postgresql://postgres:
[password]@[host]:5432/postgres
FRONTEND_URL              = https://your-app.vercel.app  (update after Vercel
deploy)
INGEST_SECRET             = any-random-secret-string-you-choose
```

Where to find these values:

- **GROQ_API_KEY**: https://console.groq.com → API Keys
- **OPENAI_API_KEY**: https://platform.openai.com → API Keys
- **SANITY_API_TOKEN**: https://sanity.io/manage → your project → API → Tokens
- **SUPABASE_URL, SUPABASE_ANON_KEY**: Supabase dashboard → Project Settings → API
- **SUPABASE_SERVICE_ROLE_KEY**: Supabase dashboard → Project Settings → API → `service_role` (keep secret!)
- **SUPABASE_JWT_SECRET**: Supabase dashboard → Project Settings → API → JWT Settings → JWT Secret
- **SUPABASE_DB_URL**: Supabase dashboard → Project Settings → Database → Connection String
  - Use the **Transaction Pooler** connection string
  - Replace `postgresql://` with `postgresql://` and add `?pgbouncer=true` at the end for pgvector compatibility
  - Full format: `postgresql://postgres.[ref]:[password]@aws-0-us-east-1.pooler.supabase.com:6543/postgres?pgbouncer=true`

> **IMPORTANT**: `SUPABASE_DB_URL` for asyncpg (used by LlamaIndex PGVectorStore) needs the **direct connection** string (port 5432), NOT the pooler. Check your backend `main.py` to confirm which port it connects on.

Step A5: Trigger the First Deploy

1. Click the **"Deploy"** tab on your Railway service
2. Click **"Deploy Now"** (or push a commit to GitHub — Railway auto-deploys on push)
3. Watch the build logs in real time
4. A successful deploy shows: `Application started production server on port XXXX`

Common build errors and fixes:

Error	Cause	Fix
`ModuleNotFoundError: No module named 'llama_index'`	Wrong root directory	Set Root Directory to `backend` in Settings
`ERROR: Could not find a version that satisfies...`	Python version mismatch	Add a `runtime.txt` with `python-3.11.0` in `backend/`
`Port already in use`	Start command uses fixed port	Ensure start command uses `$PORT` not `8000`
`Connection refused to Supabase`	Wrong DB URL	Double-check SUPABASE_DB_URL — direct vs. pooler

---

Step A6: Get Your Railway URL

1. After successful deploy, click the **"Settings"** tab
2. Under **"Networking"**, click **"Generate Domain"**
3. Railway assigns a URL like: `https://vermont-health-backend-production.up.railway.app`
4. **Copy this URL** — you need it for the Vercel configuration in the next step

Test your backend is live:

```
https://your-railway-url.railway.app/docs
```

This should show the FastAPI interactive documentation (Swagger UI).

PART B: Deploy the Frontend to Vercel

Step B1: Create a Vercel Account

1. Go to https://vercel.com
2. Click **"Sign Up"** → **"Continue with GitHub"**
3. Authorize Vercel to access your GitHub account
4. You will land on the Vercel dashboard

Step B2: Import Your GitHub Repository

1. Click **"Add New..."** → **"Project"**
2. Under **"Import Git Repository"**, find your `Vermont-Health-Platform` repository
3. Click **"Import"**

Step B3: Configure the Vercel Project

On the configuration screen:

Framework Preset: Vercel should auto-detect **Next.js** — confirm this is selected

Root Directory: This is critical.

- Click **"Edit"** next to Root Directory
- Type: `frontend`
- Click **"Continue"**

> Your `vercel.json` at the project root already sets `buildCommand: "cd frontend && npm run build"` but setting the Root Directory to `frontend` is cleaner and more reliable.

Build and Output Settings (Vercel auto-fills these from `vercel.json` — verify they are correct):

- **Build Command**: `npm run build`
- **Output Directory**: `.next`
- **Install Command**: `npm install`

Step B4: Add Frontend Environment Variables

On the configuration screen, click **"Environment Variables"** and add each one:

```
NEXT_PUBLIC_SUPABASE_URL            = https://clryhwqaqhvdikgesjbc.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY       = your-supabase-anon-key
SUPABASE_SERVICE_ROLE_KEY           = your-supabase-service-role-key
NEXT_PUBLIC_SANITY_PROJECT_ID       = fxz10xl7
NEXT_PUBLIC_SANITY_DATASET          = production
SANITY_API_TOKEN                    = your-sanity-token
PYTHON_BACKEND_URL                  = https://your-railway-url.railway.app
NEXT_PUBLIC_APP_URL                 = https://your-app.vercel.app
STRIPE_SECRET_KEY                   = sk_live_...  (or sk_test_... for testing)
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY  = pk_live_...  (or pk_test_...)
STRIPE_WEBHOOK_SECRET               = whsec_...
RESEND_API_KEY                      = re_...
```

Environment targeting: Set each variable to apply to **Production**, **Preview**, and **Development** (or use separate values for Production vs. Preview if you want a staging setup).

> **Note on NEXT_PUBLIC_APP_URL**: You don't know your exact Vercel URL yet. Set this to a placeholder (`https://placeholder.vercel.app`) for the initial deploy, then update it after Vercel assigns your URL. Redeploy after updating.

Step B5: Update `vercel.json` with Your Railway URL

Before clicking "Deploy", update the `vercel.json` in the project root:

Current `vercel.json`:

```
"rewrites": [
  {
    "source": "/api/ai/:path*",
    "destination": "https://your-backend.railway.app/api/:path*"
  }
]
```

Update to your actual Railway URL:

```
"rewrites": [
  {
    "source": "/api/ai/:path*",
    "destination": "https://vermont-health-backend-
production.up.railway.app/api/:path*"
```

```
    }
  ]
```

Also update the CORS header:

```
"headers": [
  {
    "source": "/api/(.*)",
    "headers": [
      { "key": "Access-Control-Allow-Origin", "value": "https://your-actual-app.vercel.app" },
      ...
    ]
  }
]
```

Commit and push this change to GitHub before deploying.

Step B6: Deploy

1. Click **"Deploy"**
2. Watch the build logs — the build runs `cd frontend && npm install && npm run build`
3. A successful deploy shows a green checkmark and a live URL

Common build errors and fixes:

Error	Cause	Fix
`Cannot find module '@/...'`	Wrong root directory	Confirm Root Directory is set to `frontend`
`Type error: ...`	TypeScript errors	Fix in code; or temporarily add `ignoreBuildErrors: true` in `next.config.ts`
`Module not found: 'styled-components'`	Node modules issue	Clear Vercel cache: Settings → Advanced → Clear Build Cache
`NEXT_PUBLIC_... is undefined`	Missing env var	Add missing variable in Vercel Project Settings → Environment Variables
Build times out (>45 min)	Large node_modules	Check for accidentally committed `node_modules` in git

---

Step B7: Get Your Vercel URL and Update Configurations

After deployment, Vercel assigns a URL like: `https://vermont-health-platform-abc123.vercel.app`

Or if you've added a custom domain, it will be your domain.

Update these places with your real URL:

1. **Vercel Environment Variable**: Update `NEXT_PUBLIC_APP_URL` to your real URL → redeploy
2. **Railway Environment Variable**: Update `FRONTEND_URL` to your real URL
3. **Supabase Auth Settings**: Add your Vercel URL to allowed redirect URLs:
   - Supabase Dashboard → Authentication → URL Configuration
   - **Site URL**: `https://your-app.vercel.app`
   - **Redirect URLs**: Add `https://your-app.vercel.app/auth/callback`
4. **Stripe Webhook**: Update to point to your Vercel URL (see Part C below)

PART C: Configure Stripe Webhooks

Step C1: Create a Stripe Webhook Endpoint

1. Go to https://dashboard.stripe.com
2. Navigate to **Developers** → **Webhooks**
3. Click **"Add endpoint"**
4. **Endpoint URL**: `https://your-app.vercel.app/api/stripe/webhook`
5. **Events to send** — select all of these:
   - `checkout.session.completed`
   - `customer.subscription.created`
   - `customer.subscription.updated`
   - `customer.subscription.deleted`
   - `invoice.payment_succeeded`
   - `invoice.payment_failed`
6. Click **"Add endpoint"**

Step C2: Get the Webhook Signing Secret

1. On the webhook detail page, click **"Reveal"** under **Signing secret**
2. Copy the `whsec_...` value
3. Add it to Vercel as: `STRIPE_WEBHOOK_SECRET = whsec_...`
4. Trigger a Vercel redeploy (or it will pick up on the next deploy)

PART D: Configure Supabase for Production

Step D1: Run Database Migrations

If your Supabase database has not had migrations applied, do this from your local machine:

```
# Install Supabase CLI if not already installed
npm install -g supabase

# Login
supabase login

# Link to your project (get the project ref from Supabase Dashboard → Settings
→ General)
supabase link --project-ref clryhwqaqhvdikgesjbc

# Apply all pending migrations
supabase db push
```

Step D2: Verify pgvector Extension

The RAG system requires pgvector. Verify it is enabled:

1. Supabase Dashboard → **SQL Editor**
2. Run:

```
SELECT * FROM pg_extension WHERE extname = 'vector';
```

3. If no rows returned, enable it:

```
CREATE EXTENSION vector;
```

Step D3: Configure Supabase Auth Redirect URLs

1. Supabase Dashboard → **Authentication** → **URL Configuration**
2. Set **Site URL**: `https://your-app.vercel.app`
3. Under **Redirect URLs**, add:
   - `https://your-app.vercel.app/auth/callback`
   - `https://your-app.vercel.app/reset-password`
   - `http://localhost:3000/auth/callback` (keep this for local development)
4. Click **Save**

Step D4: Configure Supabase CORS

1. Supabase Dashboard → **Settings** → **API**
2. Under **Allowed Origins** / CORS, add your Vercel domain: `https://your-app.vercel.app`

PART E: Trigger Initial RAG Knowledge Base Build

The AI Analyst cannot answer questions until the knowledge base is built (documents embedded into pgvector). Do this once after deploying:

Step E1: Trigger Ingestion

```
curl -X POST https://your-railway-url.railway.app/api/ingest \
  -H "Content-Type: application/json" \
  -H "X-Ingest-Secret: your-ingest-secret-value" \
  -d '{}'
```

Replace `your-ingest-secret-value` with the value you set for `INGEST_SECRET` in Railway.

Watch the Railway logs for ingestion progress. You should see:

```
INFO: Loading PDFs from backend/data/...
INFO: Fetching Sanity content...
INFO: Ingesting X documents...
INFO: Knowledge base build complete. X nodes indexed.
```

This may take 5–10 minutes for a full build.

Step E2: Verify the Knowledge Base

After ingestion completes, check the Supabase `rag_documents` table:

```
SELECT COUNT(*) FROM rag_documents;
SELECT DISTINCT metadata->>'source' FROM rag_documents;
```

You should see hundreds of rows and multiple source types (PDFs + Sanity content types).

PART F: Add a Custom Domain (Optional)

Step F1: Purchase or Configure Your Domain

If you have a domain (e.g., `healthtransformationreport.com`):

1. Vercel Dashboard → your project → **Settings** → **Domains**
2. Click **"Add Domain"**
3. Enter your domain: `healthtransformationreport.com`
4. Vercel provides DNS records to add:
   - **A record**: `76.76.21.21` (Vercel IP)
   - **CNAME**: for `www` subdomain → `cname.vercel-dns.com`
5. Add these records in your domain registrar's DNS settings
6. Wait for DNS propagation (5 minutes to 48 hours)
7. Vercel automatically provisions an SSL certificate via Let's Encrypt

---

Step F2: Update All URLs with Custom Domain

After the custom domain is live, update:

1. **Vercel**: `NEXT_PUBLIC_APP_URL` → `https://healthtransformationreport.com`
2. **Railway**: `FRONTEND_URL` → `https://healthtransformationreport.com`
3. **Supabase Auth**: Update Site URL and Redirect URLs
4. **Stripe Webhook**: Update endpoint URL
5. **vercel.json**: Update CORS allowed origin

PART G: Post-Deployment Verification Checklist

Work through this checklist after deploying to confirm everything is working:

Authentication

- [ ] Can create a new account at `/signup`
- [ ] Receive email verification (check spam)
- [ ] Can log in at `/login`
- [ ] Redirected to `/onboarding` after first login
- [ ] Can log out from account menu

Content

- [ ] Homepage loads with hero carousel
- [ ] Pillar pages load articles (Policy, Economics, Technology, Clinical, Equity)
- [ ] News ticker shows live headlines
- [ ] Search (⌘K) returns results

AI Analyst

- [ ] `/chat` loads for subscriber+ users
- [ ] Type a question → response streams back
- [ ] Follow-up suggestions appear
- [ ] Non-subscribers are redirected to upgrade page

Subscription & Billing

- [ ] Pricing page loads at `/pricing`
- [ ] Clicking "Subscribe" opens Stripe Checkout
- [ ] Complete a test purchase (use Stripe test card: `4242 4242 4242 4242`)
- [ ] User role upgrades to `subscriber` after checkout
- [ ] Access to subscriber-only features is granted

Research Lab

- [ ] `/research-lab` loads for professional+ users
- [ ] At least one tool (e.g., CEA Calculator) functions correctly

Backend Health

- [ ] Visit `https://your-railway-url.railway.app/docs` — Swagger UI appears
- [ ] Railway logs show no errors

Email

- [ ] Test the digest endpoint: `POST /api/digest` with test subscriber
- [ ] Email arrives in inbox from correct address

PART H: Continuous Deployment Setup

After the initial deploy, all future changes deploy automatically:

- **Push to `main`** → Vercel automatically deploys the frontend
- **Push to `main`** → Railway automatically deploys the backend
- **Preview deployments**: Every PR creates a Vercel preview URL for review before merging

To deploy manually:

```
# Frontend (from project root)
npx vercel --prod

# Backend (from project root)
railway up --service backend
```

PART I: Environment Variable Summary

Print this table and keep it with your deployment credentials:

Vercel (Frontend) Environment Variables

Variable	Description	Where to Get
`NEXT_PUBLIC_SUPABASE_URL`	Supabase project URL	Supabase → Settings → API
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Supabase anon key (public)	Supabase → Settings → API
`SUPABASE_SERVICE_ROLE_KEY`	Supabase admin key (secret!)	Supabase → Settings → API
`NEXT_PUBLIC_SANITY_PROJECT_ID`	Sanity project ID	Sanity → Manage → Project
`NEXT_PUBLIC_SANITY_DATASET`	Sanity dataset name	`production`
`SANITY_API_TOKEN`	Sanity API token	Sanity → Manage → API → Tokens
`PYTHON_BACKEND_URL`	Railway backend URL	Railway → your service → domain
`NEXT_PUBLIC_APP_URL`	Your Vercel/custom domain URL	After deploy
`STRIPE_SECRET_KEY`	Stripe secret key	Stripe → Developers → API keys
`NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`	Stripe publishable key	Stripe → Developers → API keys
`STRIPE_WEBHOOK_SECRET`	Stripe webhook signing secret	Stripe → Webhooks → endpoint

Variable	Description	Where to Get
`RESEND_API_KEY`	Resend email API key	resend.com → API Keys

Railway (Backend) Environment Variables

Variable	Description	Where to Get
`GROQ_API_KEY`	Groq LLM API key	console.groq.com → API Keys
`GROQ_MODEL`	Model name	`llama-3.3-70b-versatile`
`OPENAI_API_KEY`	OpenAI embedding key	platform.openai.com → API keys
`SANITY_PROJECT_ID`	Sanity project ID	`fxz10xl7`
`SANITY_DATASET`	Sanity dataset	`production`
`SANITY_API_TOKEN`	Sanity API token	Sanity → Manage → API → Tokens
`SUPABASE_URL`	Supabase URL	Supabase → Settings → API
`SUPABASE_SERVICE_ROLE_KEY`	Supabase admin key	Supabase → Settings → API
`SUPABASE_JWT_SECRET`	JWT secret for auth	Supabase → Settings → API → JWT
`SUPABASE_DB_URL`	Direct PostgreSQL connection	Supabase → Settings → Database
`FRONTEND_URL`	Your Vercel URL	After Vercel deploy
`INGEST_SECRET`	Secret to protect `/api/ingest`	Choose any strong random string

PART J: Troubleshooting Common Production Issues

"AI Analyst returns no results"

Cause: Knowledge base was never built, or `rag_documents` table is empty. **Fix**: Trigger ingestion manually (Part E, Step E1). Check Railway logs for ingestion errors.

"Login fails with 'Invalid redirect URL'"

Cause: Supabase doesn't recognize your production URL. **Fix**: Add your Vercel URL to Supabase Auth → URL Configuration → Redirect URLs (Part D, Step D3).

"Stripe webhook 400 error"

Cause: Webhook secret mismatch. **Fix**: Regenerate `STRIPE_WEBHOOK_SECRET` from Stripe dashboard and update in Vercel env vars.

"Backend 500 errors on Railway"

Cause: Missing environment variable or database connection failure. **Fix**: Check Railway logs. Verify all env vars are set. Test DB connection string with a psql client.

"CORS errors in browser console"

Cause: Railway backend's CORS `allow_origins` doesn't include your Vercel URL. **Fix**: Add your production URL to the `CORSMiddleware` `allow_origins` list in `backend/main.py`. Redeploy backend.

"Vercel build fails with TypeScript errors"

Cause: Type errors that don't appear in local development (e.g., stricter type checking). **Fix**: Run `npx tsc --noEmit` locally from the `frontend/` directory to see all errors before pushing.

"Content not showing (Sanity)"

Cause: Sanity API token missing or wrong dataset. **Fix**: Verify `SANITY_API_TOKEN` and `NEXT_PUBLIC_SANITY_DATASET` in Vercel env vars.

"State maps show but no data"

Cause: `state_health_metrics` table is empty or RLS is blocking access. **Fix**: Check Supabase table contents. If RLS is blocking, ensure the user is authenticated and has the correct role.

PART K: Estimated Monthly Hosting Costs

Service	Plan	Estimated Monthly Cost
Vercel	Pro (required for cron jobs, team features)	$20/month
Vercel	Hobby (free tier, no cron, 1 member)	$0/month
Railway	Hobby ($5 credit/month, then usage-based)	$5–20/month
Railway	Pro (always-on, faster cold start)	$20/month base
Supabase	Free tier (500MB DB, 2GB bandwidth)	$0/month
Supabase	Pro (8GB DB, 250GB bandwidth)	$25/month
Groq	Pay-per-token (very cheap, ~$0.05/million tokens)	$5–20/month
OpenAI	Embeddings (text-embedding-3-small, ~$0.02/million tokens)	$1–5/month
Resend	Free tier (3,000 emails/month)	$0/month
Stripe	2.9% + $0.30 per transaction	Variable
Total	MVP production setup	~$50–90/month

End of Deployment Guide

Document last updated: March 2026 Vermont Health Platform — Internal Planning Document